



EL GUARDIÁN DEL ESQUEMA Y LOS VÍNCULOS ARCANOS

Base de Datos

NoSQL

Herramienta

VSCode

Entorno

MongoDB Atlas

Autor

Pepinós Arboleda Brian



Autor

Nombre completo: TNTE. Pepinós Arboleda Brian
Asignatura: Modelado Avanzado de Base de Datos
Fecha: 26 Octubre 2025
Repositorio de GitHub:
<https://github.com/BRIANPEPINOS/mision-mongodb-pepinos>



Descripción de la Misión 2

En esta segunda misión, el Guardián del Esquema fue convocado para proteger la integridad del *Bestiario Digital*, aplicando reglas de validación y modelado avanzado de relaciones en MongoDB.

El objetivo fue crear un sistema robusto que evite la inserción de datos corruptos mediante **JSON Schema**, y que además establezca correctamente las relaciones entre entidades, tanto **embebidas** como **referenciadas**, en las colecciones **guardianes** y **criaturas**.

Esta misión se centró en blindar la base de datos contra errores, implementando contraseñas seguras, restricciones por tipo, valores únicos y relaciones 1–1 y 1–N.

Con este trabajo, se fortaleció la estructura del *Bestiario Digital*, demostrando que MongoDB puede ser tan estricto y confiable como cualquier sistema relacional cuando se diseña con validaciones adecuadas.



Estructura del Proyecto

Archivo	Descripción
 01_definicion_guardianes.mongodb	Define la colección guardianes con validación de datos: rangos, contraseñas seguras y un inventario embebido.
 02_definicion_criaturas.mongodb	Define la colección criaturas con validaciones de tipo, ficha veterinaria embebida y relación referenciada con id_guardian .
 03_pruebas_insercion.mongodb	Contiene las pruebas de inserción válidas e inválidas con los mensajes de error devueltos por MongoDB.
 ANALISIS_VALIDACION.md	Documento teórico que analiza las decisiones de validación, relaciones y ventajas del uso de JSON Schema.

Todos los archivos se ejecutan desde la carpeta **misión_02_validacion/** utilizando la extensión **MongoDB for VS Code**.

🔗 Instrucciones para ejecutar los archivos en VS Code

1. Abre **Visual Studio Code** y asegúrate de tener instalada la extensión oficial **MongoDB for VS Code**.
 2. Conéctate a tu base de datos local o a tu cluster de Atlas.
 3. Abre cada archivo `.mongodb` en el orden indicado:
 - `01_definicion_guardianes.mongodb`
 - `02_definicion_criaturas.mongodb`
 - `03_pruebas_insercion.mongodb`
 4. Verifica que aparezca en la esquina superior derecha:
☒ **Connected to Cluster / localhost**
 5. Ejecuta con el botón ► **Run Playground** o presionando `Ctrl + Shift + R`.
 6. Observa los resultados de las validaciones y errores directamente en la consola del Playground.
-

🔗 Pruebas de Integridad

Durante las pruebas, se realizaron inserciones válidas e inválidas para comprobar las validaciones aplicadas. MongoDB devolvió el código de error **121 (Document failed validation)** cuando un documento no cumplía con las reglas del esquema, confirmando que el blindaje de la base de datos funciona correctamente. Las pruebas incluyeron contraseñas no válidas, rangos fuera de los límites y arreglos vacíos en campos requeridos.

🗺️ Resultados del Modelado

- Las relaciones **embebidas** (Guardián → Inventario y Criatura → Ficha Veterinaria) mejoraron la coherencia y las operaciones atómicas.
 - Las relaciones **referenciadas** (Guardián → Criaturas) optimizaron el rendimiento y la escalabilidad del sistema.
 - El uso de **JSON Schema** permitió validar tipos, rangos, enumeraciones y patrones de texto directamente desde la base de datos.
 - Los errores de validación fueron gestionados de forma inmediata, demostrando que el esquema protege los datos ante entradas incorrectas.
-

📄 Conclusión

La misión 2 demostró que MongoDB no solo es una base de datos flexible, sino también segura y confiable cuando se combina con validación estructural y modelado adecuado.

El Guardián del Esquema logró su propósito: proteger la pureza del *Bestiario Digital* asegurando que cada documento mantenga su forma, reglas y vínculos dentro del universo de datos.

El proyecto consolidó buenas prácticas de diseño, uso de `validator: $jsonSchema`, manejo de relaciones y ejecución de pruebas controladas desde Visual Studio Code, fortaleciendo así la comprensión del modelo NoSQL con enfoque profesional.
